

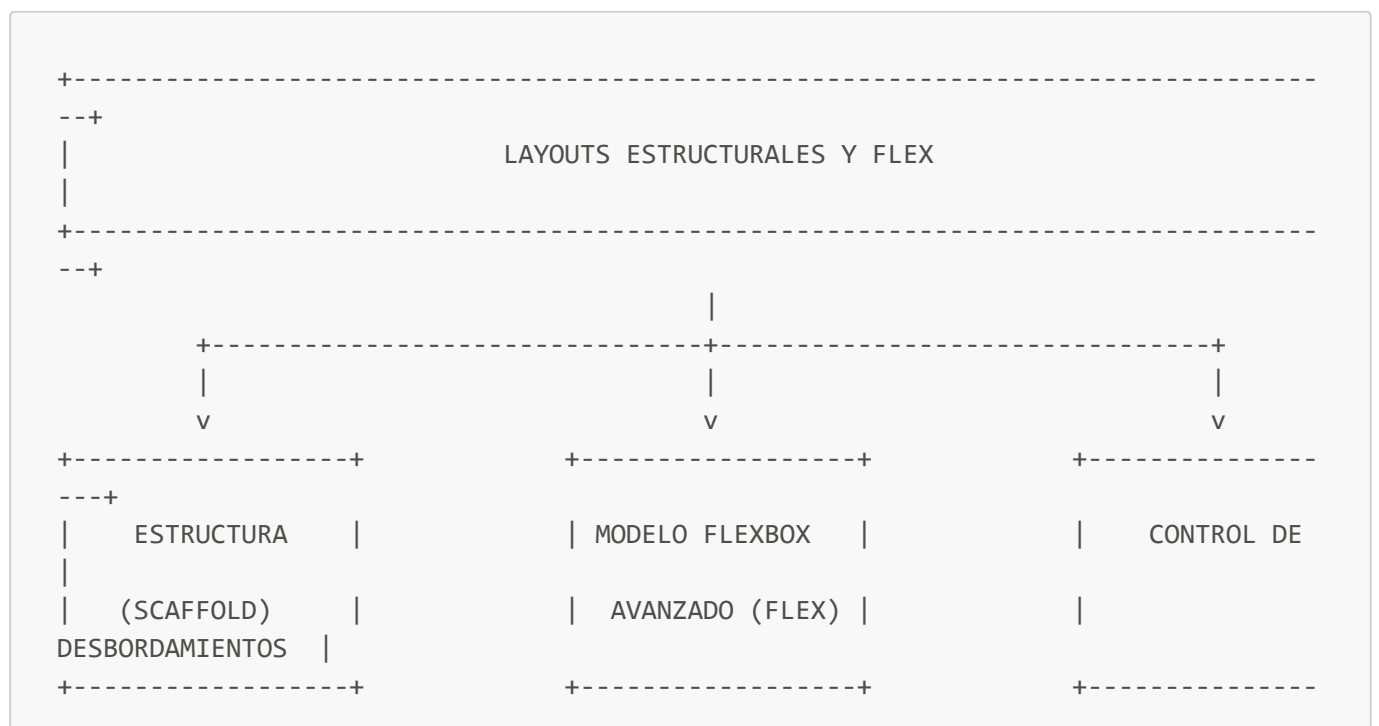
Introducción

Uno de los principales retos al construir interfaces responsivas en Flutter es la gestión de las restricciones de pantalla (*constraints*). Dado que Flutter utiliza un modelo de maquetación estricto donde **"las restricciones bajan, los tamaños suben y los padres posicionan"**, un mal cálculo del espacio disponible por parte de contenedores flexibles (**Row**, **Column**, **Flex**) genera inevitablemente errores de renderizado. El más común y notorio de ellos es el desbordamiento visual (*RenderFlex overflow*), caracterizado por la icónica franja diagonal amarilla y negra.

En esta sesión profundizaremos en el armazón estructural predeterminado del framework (**Scaffold**), el funcionamiento detallado del modelo Flexbox en Dart (**Row**, **Column**, **Flex**, **Expanded**, **Flexible**, **Spacer**) y las estrategias técnicas de ingeniería para diagnosticar, corregir y prevenir desbordamientos de pantalla en escenarios multi-dispositivo.

Conceptos y Definiciones

Para comprender la arquitectura del diseño estructural en Flutter, debemos analizar sus componentes principales desde la mecánica del *Layout Phase*.



```

---+
| • AppBar          | | • MainAxis          | | •
SingleChild... |
| • Body            | | • CrossAxis         | | • ListView
|
| • Drawer          | | • Expanded          | | • FittedBox
|
| • FAB             | | • Flexible          | | •
LayoutBuilder  |
| • BottomNav       | | • Spacer            | | • Wrap
|
+-----+          +-----+          +-----+
---+

```

1. El Armazón Estructural: **Scaffold**

El widget **Scaffold** implementa la estructura visual básica de maquetación siguiendo las guías de Material Design. Actúa como el contenedor principal de una pantalla y proporciona ranuras (*slots*) predefinidas para ubicar componentes esenciales sin necesidad de calcular manualmente su posición en la pantalla:

- **appBar**: Acepta un widget que implemente **PreferredSizeWidget** (habitualmente **AppBar**) para la barra superior de acciones y título.
- **body**: El área principal de contenido de la pantalla, ubicada debajo de la **appBar** y por encima de la barra inferior.
- **drawer / endDrawer**: Paneles de navegación desglosables deslizables desde el borde izquierdo o derecho respectivamente.
- **floatingActionButton (FAB)**: Botón de acción principal flotante que se superpone en la esquina inferior del **body**.
- **bottomNavigationBar / bottomSheet**: Barras de navegación inferior o paneles persistentes acoplados a la parte baja de la vista.
- **resizeToAvoidBottomInset**: Booleano crítico que determina si el **body** debe redimensionarse automáticamente para evitar ser solapado cuando el teclado software en pantalla se despliega.

2. El Modelo Flexbox en Flutter: **Flex**, **Row** y **Column**

El modelo de maquetación en una sola dimensión en Flutter está respaldado por la clase base **Flex**. De hecho, **Row** y **Column** son clases derivadas directas de **Flex**:

- **Row**: Hereda de **Flex** fijando la propiedad **direction: Axis.horizontal**.
- **Column**: Hereda de **Flex** fijando la propiedad **direction: Axis.vertical**.

Propiedades Eje Principal y Cruzado

- **MainAxisAlignment**: Define cómo se distribuye el espacio sobrante en el eje principal entre o alrededor de los hijos (**start**, **end**, **center**, **spaceBetween**, **spaceAround**, **spaceEvenly**).
- **CrossAxisAlignment**: Define la alineación de los hijos a lo largo del eje perpendicular (**start**, **end**, **center**, **stretch**, **baseline**). La propiedad **stretch** fuerza a todos los hijos a expandirse para llenar

completamente el ancho/alto del eje cruzado.

Asignación Proporcional de Espacio: **Expanded**, **Flexible** y **Spacer**

- **Expanded**: Un widget wrapper que fuerza a su hijo a ocupar **todo el espacio disponible restante** a lo largo del eje principal. Acepta un parámetro **flex** (entero) que actúa como un factor de peso para repartir el espacio disponible entre múltiples hermanos flexibles.
 - **Flexible**: Proporciona flexibilidad al hijo pero, a diferencia de **Expanded**, no lo obliga a llenar todo el espacio asignado. Utiliza la propiedad **fit**:
 - **FlexFit.tight** (Comportamiento idéntico a **Expanded**): Obliga al hijo a expandirse al máximo espacio asignado.
 - **FlexFit.loose**: Permite al hijo ser tan pequeño como desee dentro del límite máximo asignado.
 - **Spacer**: Es un widget utilitario equivalente a **Expanded(child: SizedBox.shrink())**. Consume espacio vacío dentro de un **Row** o **Column** utilizando el factor **flex**.
-

3. Anatomía y Prevención del Desbordamiento (*RenderFlex Overflow*)

El desbordamiento ocurre cuando la suma de los tamaños no flexibles solicitados por los hijos de un **Row** o **Column** excede el tamaño máximo permitido por las restricciones del padre en el eje principal.

Mapeo de Soluciones Técnicas para Desbordamiento

- **SingleChildScrollView**: Envuelve al contenedor flex para convertir el eje en una superficie desplazable (*scrollable*). Ideal para formularios o contenido cuya altura total varía en pantallas pequeñas.
 - **FittedBox**: Escala y escala de forma proporcional (*scale down*) a su hijo para forzarlo a encajar dentro del espacio asignado sin desbordar.
 - **Wrap**: Widget sustituto de **Row** o **Column** que rompe el contenido en múltiples líneas o columnas (salto de línea automático) cuando se agota el espacio en el eje principal.
 - **LayoutBuilder**: Permite inspeccionar las restricciones entrantes del padre (**BoxConstraints**) en tiempo de renderizado para tomar decisiones condicionales sobre qué widget renderizar según el tamaño disponible.
-

Ejemplo Completo con Código Fuente: Consola de Gestión Estudiantil

El siguiente ejemplo integra la estructura completa de un **Scaffold** con un drawer de navegación, junto con un layout receptivo que utiliza **Row**, **Column**, **Expanded**, **Spacer** y manejo explícito de desbordamientos mediante **SingleChildScrollView** y **FittedBox**.

Código Fuente Explicado (**lib/main.dart**)

```
import 'package:flutter/material.dart';

void main() {
```

```
runApp(const DemoLayoutsEstructuralesApp());
}

/// Widget Raíz de la Aplicación
class DemoLayoutsEstructuralesApp extends StatelessWidget {
  const DemoLayoutsEstructuralesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Layouts Estructurales',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaGestionEstudiantil(),
    );
  }
}

/// Pantalla Principal basada en Scaffold
class PantallaGestionEstudiantil extends StatefulWidget {
  const PantallaGestionEstudiantil({super.key});

  @override
  State<PantallaGestionEstudiantil> createState() =>
    _PantallaGestionEstudiantilState();
}

class _PantallaGestionEstudiantilState
  extends State<PantallaGestionEstudiantil> {
  int _contadorRegistros = 120;
  bool _mostrarFalloDesbordamiento = false;

  void _incrementarRegistros() {
    setState(() {
      _contadorRegistros += 5;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // 1. ESTRUCTURA: BARRA SUPERIOR
      appBar: AppBar(
        title: const Text('Gestión de Alumnos ESCOM'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        actions: [
          IconButton(
            icon: const Icon(Icons.tune),

```

```

        tooltip: 'Simular Desbordamiento',
        onPressed: () {
          setState(() {
            _mostrarFalloDesbordamiento = !_mostrarFalloDesbordamiento;
          });
        },
      ),
    ],
  ),
),

// 2. ESTRUCTURA: NAVEGACIÓN LATERAL (DRAWER)
drawer: Drawer(
  child: ListView(
    padding: EdgeInsets.zero,
    children: <Widget>[
      DrawerHeader(
        decoration: BoxDecoration(
          color: Theme.of(context).colorScheme.primary,
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.center,
          children: const <Widget>[
            CircleAvatar(
              radius: 30,
              backgroundColor: Colors.white,
              child: Icon(Icons.school, size: 35, color: Color(0xFF6B112D)),
            ),
            SizedBox(height: 10),
            Text(
              'Sistema de Control Escolar',
              style: TextStyle(color: Colors.white, fontSize: 16,
fontWeight: FontWeight.bold),
            ),
            Text(
              'escom@ipn.mx',
              style: TextStyle(color: Colors.white70, fontSize: 12),
            ),
          ],
        ),
      ),
      ListTile(
        leading: const Icon(Icons.dashboard),
        title: const Text('Panel Principal'),
        onTap: () => Navigator.pop(context),
      ),
      ListTile(
        leading: const Icon(Icons.people),
        title: const Text('Inscripciones'),
        onTap: () => Navigator.pop(context),
      ),
      const Divider(),
      ListTile(
        leading: const Icon(Icons.settings),

```

```

        title: const Text('Configuración'),
        onTap: () => Navigator.pop(context),
      ),
    ],
  ),
),

// 3. ESTRUCTURA: CUERPO PRINCIPAL (BODY) CON MANEJO DE DESBORDAMIENTO
body: SingleChildScrollView(
  padding: const EdgeInsets.all(16.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: <Widget>[
      // Banner de Advertencia de Estado
      if (_mostrarFalloDesbordamiento)
        Container(
          margin: const EdgeInsets.only(bottom: 16.0),
          padding: const EdgeInsets.all(12.0),
          decoration: BoxDecoration(
            color: Colors.amber.shade100,
            border: Border.all(color: Colors.amber.shade900),
            borderRadius: BorderRadius.circular(8.0),
          ),
          child: Row(
            children: const <Widget>[
              Icon(Icons.warning, color: Colors.amber),
              SizedBox(width: 10),
              Expanded(
                child: Text(
                  'Modo de Prueba: Se ha forzado un Row sin Expanded abajo
para evidenciar o prevenir el RenderFlex Overflow.',
                  style: TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),
                ),
              ),
            ],
          ),
        ),
      // SECCIÓN 1: CABECERA FLEXIBLE CON SPACER Y EXPANDED
      const Text(
        'Estadísticas del Semestre Activo',
        style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
      ),
      const SizedBox(height: 12.0),
      Row(
        children: <Widget>[
          // Widget Flex con peso flex: 2
          Expanded(
            flex: 2,
            child: _buildTarjetaStat(
              'Inscritos',
              '$_contadorRegistros',

```

FILAS

```

        Colors.blue.shade800,
        Icons.assignment_ind,
      ),
    ),
    const SizedBox(width: 8.0),
    // Widget Flex con peso flex: 1
    Expanded(
      flex: 1,
      child: _buildTarjetaStat(
        'Grupos',
        '18',
        Colors.green.shade800,
        Icons.class_,
      ),
    ),
  ],
),
const SizedBox(height: 20.0),

// SECCIÓN 2: DEMOSTRACIÓN DE CONTROL DE DESBORDAMIENTO EN TEXTOS Y
const Text(
  'Detalle de la Unidad de Aprendizaje',
  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
),
const SizedBox(height: 12.0),

Card(
  elevation: 2,
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      children: <Widget>[
        // DEMOSTRACIÓN: Manejo de desbordamiento de texto horizontal
        _mostrarFalloDesbordamiento
        ? // Fila con desbordamiento (Falla intencional)
        Row(
          children: const <Widget>[
            Icon(Icons.book, color: Color(0xFF6B112D)),
            SizedBox(width: 10),
            Text(
              'Laboratorio de Desarrollo de Aplicaciones Móviles
Multiplataforma para Ingeniería',
              style: TextStyle(fontSize: 16, fontWeight:
FontWeight.bold),
            ),
          ],
        )
        : // Fila Corregida con Expanded para evitar Overflow
        Row(
          children: <Widget>[
            const Icon(Icons.book, color: Color(0xFF6B112D)),
            const SizedBox(width: 10),
            Expanded(

```

```

        child: Text(
          'Laboratorio de Desarrollo de Aplicaciones
Móviles Multiplataforma para Ingeniería',
          style: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
          overflow: TextOverflow.ellipsis,
          maxLines: 2,
        ),
      ),
    ],
  ),
),

const Divider(height: 24),

// USO DE SPACER Y FITTEDBOX
Row(
  children: <Widget>[
    const Text(
      'Cupo Total:',
      style: TextStyle(color: Colors.grey),
    ),
    const SizedBox(width: 5),
    // FittedBox previene que un texto numérico grande rompa
    el espacio

    const FittedBox(
      fit: BoxFit.scaleDown,
      child: Text(
        '40 Alumnos',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
    ),
    // Spacer empuja los siguientes elementos al extremo
    derecho

    const Spacer(),
    Chip(
      avatar: const Icon(Icons.check_circle, color:
Colors.green, size: 18),
      label: const Text('Aprobado'),
      backgroundColor: Colors.green.shade50,
    ),
  ],
),
],
),
),
),
const SizedBox(height: 20.0),

// SECCIÓN 3: USO DE FLEXFIT.LOOSE VS FLEXFIT.TIGHT
const Text(
  'Distribución de Carga Docente (Flexible vs Expanded)',
  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),

```

```

    ),
    const SizedBox(height: 12.0),

    Container(
      padding: const EdgeInsets.all(12),
      color: Colors.grey.shade200,
      child: Row(
        children: <Widget>[
          // Flexible con FlexFit.loose: solo ocupa lo que necesita su
contenido
          Flexible(
            fit: FlexFit.loose,
            child: Container(
              padding: const EdgeInsets.all(8),
              color: Colors.orange.shade300,
              child: const Text('Loose (Ajustado)'),
            ),
          ),
          const SizedBox(width: 10),
          // Expanded (FlexFit.tight): obliga a tomar todo el espacio
restante
          Expanded(
            child: Container(
              padding: const EdgeInsets.all(8),
              color: Colors.teal.shade300,
              child: const Text(
                'Tight / Expanded (Toma todo el resto)',
                textAlign: TextAlign.center,
              ),
            ),
          ),
        ],
      ),
    ),
  ],
),
),

// 4. ESTRUCTURA: BOTÓN FLOTANTE (FAB)
floatingActionButton: FloatingActionButton.extended(
  onPressed: _incrementarRegistros,
  backgroundColor: Theme.of(context).colorScheme.primary,
  foregroundColor: Colors.white,
  icon: const Icon(Icons.add),
  label: const Text('Agregar Alumno'),
),

// 5. ESTRUCTURA: BARRA DE NAVEGACIÓN INFERIOR
bottomNavigationBar: BottomNavigationBar(
  currentIndex: 0,
  selectedItemColor: Theme.of(context).colorScheme.primary,
  items: const [
    BottomNavigationBarItem(
      icon: Icon(Icons.home),

```

```

        label: 'Inicio',
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.school),
        label: 'Cursos',
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.person),
        label: 'Perfil',
      ),
    ],
  ),
);
}

/// Método auxiliar para construir tarjetas estadísticas
Widget _buildTarjetaStat(String titulo, String valor, Color color, IconData
icono) {
  return Container(
    padding: const EdgeInsets.all(16.0),
    decoration: BoxDecoration(
      color: color.withOpacity(0.1),
      borderRadius: BorderRadius.circular(12.0),
      border: Border.all(color: color.withOpacity(0.3)),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: <Widget>[
            Text(
              titulo,
              style: TextStyle(color: color, fontWeight: FontWeight.w600),
            ),
            Icon(icono, color: color, size: 20),
          ],
        ),
        const SizedBox(height: 8.0),
        Text(
          valor,
          style: TextStyle(
            fontSize: 22.0,
            fontWeight: FontWeight.bold,
            color: color,
          ),
        ),
      ],
    ),
  );
}
}

```

Prácticas Guiadas / Laboratorio

Práctica 1: Reproducción y Diagnóstico del Error *RenderFlex Overflow*

1. Ejecuta el código proporcionado y presiona el icono de sintonización (**Icons.tune**) ubicado en la esquina superior derecha del **AppBar** para activar el estado de error.
 2. Observa cómo la pantalla muestra una barra amarilla y negra indicando un desbordamiento hacia la derecha por un número específico de píxeles (*RIGHT OVERFLOWED BY X PIXELS*).
 3. Inspecciona el código correspondiente en el método **build** dentro de la tarjeta de detalle de la unidad de aprendizaje.
 4. Identifique la razón: El widget **Text** tiene una cadena larga de texto pero está anidado directamente en un **Row** sin restricciones de ancho. Al no tener restricciones, el **Text** solicita al **Row** todo el ancho necesario para renderizarse en una sola línea, excediendo el límite de la pantalla.
 5. Cambie la bandera **_mostrarFalloDesbordamiento** para observar la solución técnica: envolver el **Text** dentro de un widget **Expanded** e incluir la propiedad **overflow: TextOverflow.ellipsis**.
-

Práctica 2: Construcción Receptiva usando **LayoutBuilder**

Desarrolla una vista que altere su disposición interna dinámicamente según el ancho de la pantalla (modo retrato vs modo paisaje):

1. Envuelve el **body** de un **Scaffold** dentro de un widget **LayoutBuilder**.
 2. Lee las restricciones entrantes mediante el parámetro **constraints.maxWidth**.
 3. Establece una regla condicional:
 - Si **constraints.maxWidth < 600** (pantalla móvil estándar), renderiza los componentes de estadísticas verticalmente usando una **Column**.
 - Si **constraints.maxWidth >= 600** (pantalla de tablet o dispositivo en modo horizontal), renderiza los componentes horizontalmente usando un **Row** con dos widgets **Expanded**.
-

Resumen y Puntos Clave

En esta sesión hemos analizado a fondo la estructura de diseño y el comportamiento de los contenedores flexibles en Flutter:

- **Estructura Estándar con **Scaffold**:** Proporciona un patrón de maquetación listo para producción con ranuras para **AppBar**, **Drawer**, **body**, **FloatingActionButton** y **BottomNavigationBar**.
- **Mecánica Flexbox (**Flex**, **Row**, **Column**):** **Row** y **Column** extienden de **Flex**. Sus propiedades **mainAxisAlignment** y **crossAxisAlignment** rigen la distribución espacial en los ejes principal y cruzado.
- **Control de Espacio Proporcional:** **Expanded** obliga al hijo a llenar todo el espacio restante (peso asignado por **flex**). **Flexible** permite que el hijo ocupe hasta un espacio máximo sin forzar el llenado si se usa con **FlexFit.loose**. **Spacer** consume espacio vacío entre elementos.
- **Mitigación de Desbordamientos (**RenderFlex Overflow**):**
- Para listas de longitud indefinida o formularios: **SingleChildScrollView** o **ListView**.

- Para elementos flexibles dentro de filas/columnas: **Expanded** o **Flexible**.
 - Para textos largos en filas: **Expanded** combinado con **TextOverflow.ellipsis**.
 - Para ajustar tamaño visual manteniendo proporciones: **FittedBox**.
 - Para reestructuración adaptativa: **LayoutBuilder** o **Wrap**.
-

Bibliografía y Referencias

1. **Google LLC.** (2024). *Layout in Flutter: Building adaptive and responsive UIs*. Recuperado de: <https://docs.flutter.dev/ui/layout>
2. **Flutter Engineering Team.** (2023). *Scaffold class documentation*. Recuperado de: <https://api.flutter.dev/flutter/material/Scaffold-class.html>
3. **Napier, R.** (2021). *Flutter In Practice: Advanced Layout Concepts and Box Constraints*. Pragmatic Programmers. ISBN: 978-1680507850.
4. **Payne, R.** (2022). *Beginning App Development with Flutter: Build Cross-Platform Mobile Apps* (Capítulo 5). Apress. ISBN: 978-1484271827.
5. **Windmill, E.** (2020). *Flutter in Action* (Capítulo 5: Layouts and Structure). Manning Publications. ISBN: 978-1617296147.